

A Comparative Evaluation of Statistical, Machine Learning, and Reinforcement Learning Models for U.S. Equity Forecasting and Trading

Jamie Dinh, Jackson Rusconi, Vanya Murzin

December 1, 2025

Abstract

This project evaluates how well classical statistical forecasting, supervised machine learning, and reinforcement learning (RL) can model and trade U.S. equities. Using daily OHLCV data for eleven stocks across technology and industrial sectors from the Kaggle Stock Market Data repository [7], we engineered a consistent set of technical indicators (returns, log returns, moving averages, rolling volatility, RSI, and volume-based features) and evaluated all models under a shared proportional split (70% training, 10% validation, 20% testing) applied in chronological order to avoid lookahead bias. Traditional predictive models (linear regression, logistic regression, random forest, and neural networks) were implemented as supervised baselines, ARIMA served as a classical time-series comparator, and RL agents (RRL and PPO) modeled trading as sequential decision-making. Exploratory analysis revealed clear sector differences: technology firms showed higher volatility and stronger momentum patterns, while industrial firms displayed steadier RSI and smoother trends, supporting diversification benefits. Among supervised models, linear and logistic regression provided in-

interpretable baselines that tracked broad trends but struggled to deliver strong daily directional signal, while Random Forest regression achieved extremely high short-horizon predictive fit (RMSE 3.2556, MAE 2.1440, MAPE 1.8524%, R^2 0.9973), contrasted by Random Forest classification performance near chance (accuracy 0.4949, ROC-AUC 0.5074), highlighting the difference between price tracking and actionable direction prediction. A profit-aligned Neural Network trained with a PnL-based objective produced the strongest supervised trading outcomes, combining high classification quality (accuracy 0.9364, F1 0.9401) with exceptional risk-adjusted performance (annualized Sharpe 13.84 and minimal drawdown). Rolling ARIMA forecasts on a subset of tickers yielded moderate one-step prediction accuracy (e.g., AAPL MSE 7.8846, MAE 1.6977), with stronger performance on lower-volatility assets. PPO trading agents achieved strong out-of-sample returns on several momentum-driven tickers (e.g., AMD +441.63%, TXN +58.22%) while underperforming on others (e.g., NVDA -19.38%, GE -62.13%), illustrating both the promise and instability of RL in noisy financial environments. Overall, ensemble and statistical models offered strong predictive baselines, but the largest and most consistent trading gains emerged when model objectives were aligned with economic reward, as seen in the PnL-optimized Neural Net and RL agents.

1 Introduction / Background

Predicting stock market movements has been a central problem in quantitative finance for decades because financial time series are noisy, non-stationary, and shaped by evolving macroeconomic and firm-level factors. Reliable forecasts matter not only for speculative trading but also for long-horizon portfolio management and risk control, where small improvements in decision quality can compound into meaningful financial outcomes. Traditional statistical approaches such as ARIMA and related econometric models attempt to capture temporal autocorrelation and trend structure, but they often struggle when markets exhibit

nonlinear behavior or sudden regime shifts. This limitation has led to the rapid adoption of machine learning methods, which can learn complex patterns directly from data without requiring explicit functional assumptions [10, 8, 12]. More recently, reinforcement learning has emerged as a promising alternative because it frames trading as sequential decision-making under uncertainty, allowing an agent to optimize cumulative reward tied directly to realized returns or risk-adjusted objectives rather than forecast error alone [6, 5, 4, 2].

The literature suggests that supervised machine learning models frequently achieve high predictive accuracy on short-horizon stock movements, especially when leveraging technical indicators, tree ensembles, or deep neural networks [10, 8]. However, multiple studies emphasize that prediction accuracy does not necessarily translate into profitable trading once transaction costs, slippage, and market frictions are accounted for [1, 3]. Reinforcement learning attempts to address this gap by learning policies that directly maximize a trading objective, and modern policy-gradient methods such as PPO have shown improved stability over earlier RL approaches in non-stationary environments [4, 2]. Even so, RL remains computationally demanding and sensitive to reward design, and it risks overfitting to historical regimes that may not persist out of sample [5, 4]. What remains comparatively underexplored is a controlled, side-by-side evaluation of classical statistical baselines, supervised ML models, and RL agents under identical data, costs, and temporal splits, particularly across multiple equity sectors [1, 5].

Motivated by that gap, this project conducts a unified comparison of statistical, supervised, and reinforcement learning paradigms for forecasting and trading U.S. equities. We use Kaggle’s Stock Market Data collection, which provides daily open, high, low, close, adjusted close, and volume values for thousands of U.S. stocks [7]. To ensure sector diversity and capture contrasting market dynamics, we selected eleven representative companies spanning technology and industrial/logistics areas: Apple (AAPL), Advanced Micro Devices (AMD), Cisco (CSCO), Microsoft (MSFT), Nvidia (NVDA), and Texas Instruments (TXN)

for technology, and Emerson Electric (EMR), General Electric (GE), Honeywell (HON), IBM, and UPS for industrial and logistics exposure. This mix enables direct comparison between high-growth, high-volatility technology firms and steadier industrial firms, setting an empirical foundation for evaluating how model families behave under different volatility and trend regimes.

Our overall objective is to evaluate whether reinforcement learning provides measurable advantages over supervised forecasting models in realistic trading settings. Specifically, we investigate three related questions. First, we ask how traditional supervised learning models such as regression, random forests, and neural networks compare to RL methods in identifying short-term and long-term patterns in stock prices and returns. Second, we examine whether RL can incorporate risk management mechanisms more naturally through its reward structure, potentially learning behaviors such as cutting losses or adjusting exposure without requiring post hoc rules. Third, we analyze the trade-off between forecast accuracy, realized profitability, and risk once realistic market frictions are included, recognizing that highly accurate models can still lose money in practice [10, 1, 6, 11].

To answer these questions, we follow a staged project plan. We first clean and merge multi-ticker OHLCV data into a consistent panel and engineer standard technical indicators. We then conduct EDA to understand volatility structure, momentum behavior, and cross-asset relationships across sectors. Next, we train and validate ARIMA models, supervised ML baselines, and RL trading agents using a walk-forward temporal split to avoid lookahead bias. Finally, we evaluate the models on predictive performance and out-of-sample trading outcomes, interpret their strengths and failures in light of existing research, and discuss practical and ethical implications for algorithmic trading.

2 Methods

2.1 Data Collection, Preprocessing and Feature Engineering

We began by exploring the raw stock market data to understand basic behavior and to decide which features would be most informative for modeling. Using the Kaggle Stock Market Data repository [7], we loaded daily OHLCV (open, high, low, close, adjusted close, volume) histories for our eleven selected tickers and computed descriptive statistics for each series. Summary tables of prices, returns, and volumes revealed clear differences between the technology and industrial/logistics stocks: technology tickers (AAPL, AMD, CISCO, MSFT, NVDA, TXN) showed higher average returns and much larger standard deviations, while industrial names (EMR, GE, HON, IBM, UPS) were less volatile and more mean-reverting. Histograms of daily returns confirmed that most return distributions were approximately centered at zero with moderate tails, but some industrial tickers exhibited occasional large shocks, which has also been observed in prior work on equity risk and downside events [1].

Time-series plots of closing prices, overlaid with short- and medium-horizon moving averages, were used to examine trend and momentum. We focused on 7-day and 30-day moving averages, which highlight short-term swings versus smoother medium-term trends. Technology tickers frequently displayed long stretches where the short moving average stayed above the long moving average, indicating persistent momentum phases, whereas some industrial tickers showed flatter trajectories with frequent moving-average crossovers. These behaviors are consistent with earlier studies that report stronger momentum and trend-following patterns in growth stocks than in mature industrial firms [10, 8]. We also plotted trading volume alongside price for each ticker to see how liquidity and price moves interact. Spikes in volume often coincided with large price changes, suggesting that volume-based features might help identify breakouts or reversals.

Based on these EDA findings and prior literature on technical-indicator-based stock

prediction [10, 8, 12], we built a common feature set for all models. From prices we computed daily simple returns and log returns, which provide scale-free measures of change and are standard inputs for risk and forecasting tasks. Moving averages over 7 and 30 days were added to capture short- and medium-term trend information. A 20-day rolling standard deviation of returns served as a local volatility estimate, summarizing recent risk. We also calculated the 14-day Relative Strength Index (RSI14) as a nonlinear momentum indicator, and included daily volume and volume changes to reflect trading activity. Calendar effects such as day-of-week and sector membership (technology vs industrial/logistics) were one-hot encoded so that learning algorithms could detect systematic patterns tied to trading days or sectors. These engineered features created a shared representation that every model family used, with the exception of ARIMA, which operates directly on the closing price series.

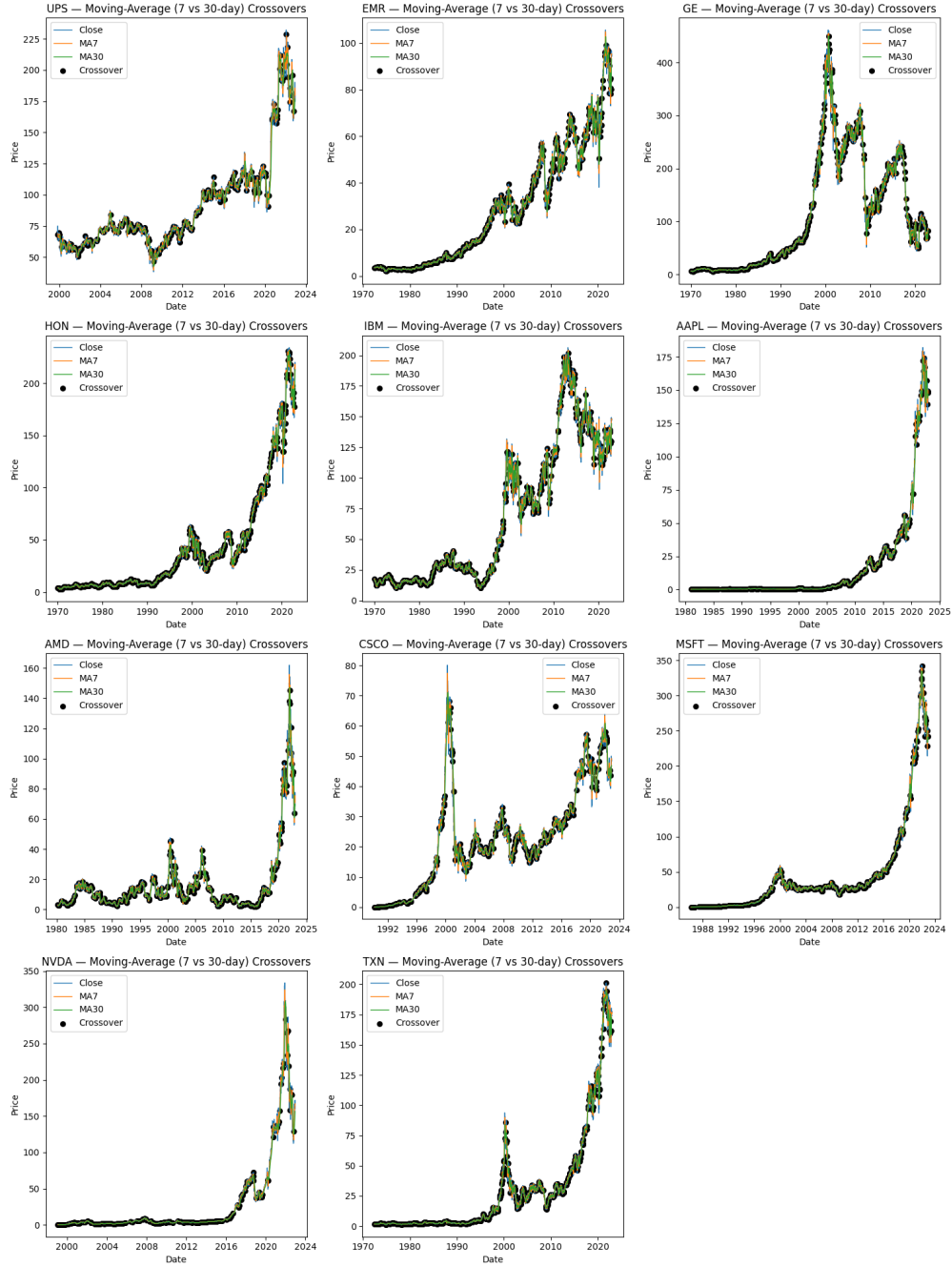


Figure 1: Closing price series with 7-day and 30-day moving averages. The technology stock exhibits stronger sustained momentum, while the industrial stock shows flatter trends and more frequent crossovers.

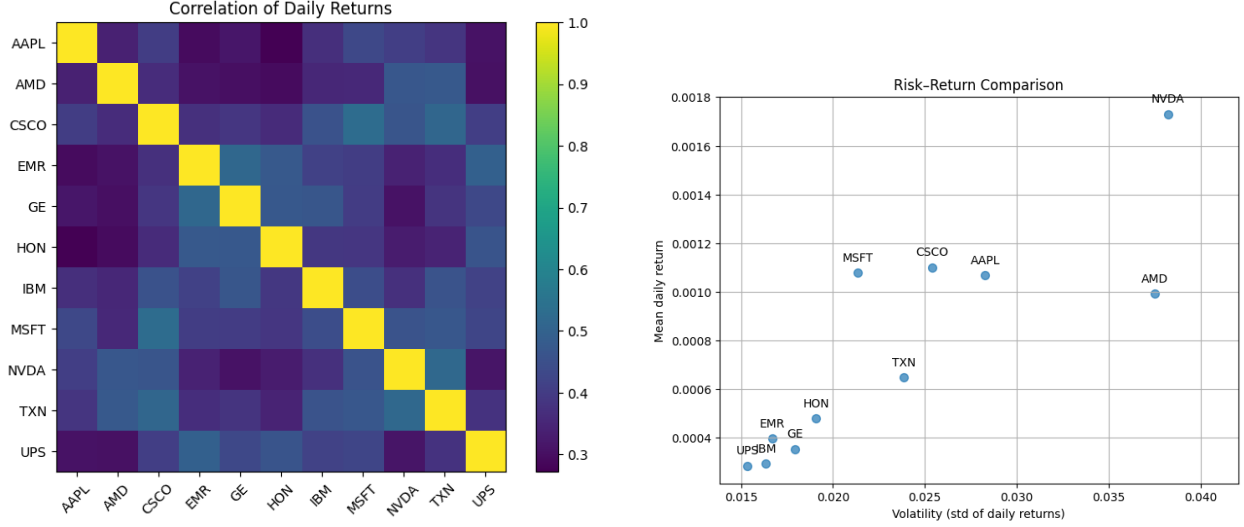


Figure 2: Left: Correlation matrix of daily returns across the eleven tickers, highlighting strong co-movement within the technology sector. Right: risk-return scatter plot showing average daily return versus volatility for each ticker; technology names cluster in a higher-risk, higher-return region relative to industrial/logistics stocks.

2.2 Data Cleaning and Integration

Before modeling, we performed several data cleaning steps to construct a consistent panel. For each ticker, the Kaggle CSV file was read into a Pandas DataFrame, columns were renamed to a common OHLCV schema, and date columns were converted to Python datetime objects. We removed obvious duplicates and sorted each ticker’s data chronologically to preserve temporal order. Because stock markets do not trade every calendar day, some tickers had missing dates relative to others; rather than forcing a common calendar with artificial rows, we allowed each series to keep its natural trading calendar and handled missing values within each ticker independently, which is standard practice in empirical finance when working with individual equities [1].

Rolling computations such as moving averages, volatility, and RSI introduce missing values at the start of each window. These initial NaNs were dropped so that all feature vectors correspond to days with complete information. For rare gaps in the raw OHLCV

data, we used forward filling within each ticker so that models would not encounter missing values during training or evaluation. After cleaning, the individual ticker tables were concatenated into a single long-format DataFrame indexed by (ticker, date), which simplifies later operations such as splitting, scaling, and model-specific preprocessing.

2.3 Data Splitting and Feature Scaling

To evaluate all models under the same conditions, we applied a consistent train/validation/test split for each ticker. For a given stock, its time-ordered history was divided into three contiguous segments: the first 70% of observations formed the training set, the next 10% formed the validation set used for hyperparameter tuning and early stopping, and the final 20% served as an unseen test set. Because the split is chronological, models are always trained in the past and evaluated in the future, which avoids the lookahead bias and better mimics a real forecasting or trading scenario, a point emphasized in prior work on machine learning for finance [10, 1].

Feature scaling was handled in a model-aware way. For the classical statistical and supervised learning models (Linear Regression, Logistic Regression, Random Forests, and the profit-aligned Neural Network), we standardized the engineered features using statistics computed on the training data only (mean and standard deviation for each feature). The same transformation was then applied to validation and test data. This standardization ensures that features with different units (for example, price levels and RSI) are placed on comparable scales without leaking information from the future into training. For the reinforcement learning agents, which are implemented with neural networks and optimized using gradient-based methods, we instead used a MinMaxScaler to map each feature into the $[0, 1]$ range on the training segment; similar min-max scaling is common in deep RL trading systems for numerical stability [5, 4]. ARIMA models, which are defined directly on price levels or differences, used the raw closing price series without additional scaling, aside from

the differencing required to achieve approximate stationarity [9].

2.4 Data Preparation and Environment Setup for Reinforcement Learning

To train reinforcement learning agents, we transformed the cleaned and scaled feature panel into sequences suitable for a recurrent policy network and defined a trading environment that mimics portfolio dynamics. For each ticker, we took the MinMax-scaled feature matrix (twelve inputs: OHLC, adjusted close, volume, return, log return, MA7, MA30, rolling volatility, and RSI14) and segmented it into overlapping windows of fixed length so that each state seen by the agent consisted of a recent history of market observations rather than a single day. The corresponding unscaled closing prices were stored separately for use in computing actual portfolio values. This design follows prior recurrent RL work in finance, where sequence information is critical for capturing temporal patterns in returns [6, 5].

The trading environment followed the standard OpenAI Gym pattern. At each time step the state consisted of the most recent feature window plus the agent’s current position (long, flat, or short). The action space contained three discrete actions: open or maintain a long position, move to cash (flat), or open or maintain a short position. An episode began with an initial cash balance of \$10,000 and no holdings. When the agent bought or sold, its position and cash were updated using the current closing price, and a proportional transaction cost of 0.1% was deducted to simulate realistic trading fees and slippage. The reward at each step was defined as the change in total net worth (cash plus the market value of any position) relative to the previous step. Episodes proceeded sequentially through the 70% training segment; trajectories collected from this interaction were used to update the agent’s policy and value networks using a PPO-style clipped policy-gradient objective [11, 4, 2].

2.5 Modeling Approaches

Our modeling strategy combined classical statistical baselines, supervised machine learning models, and reinforcement learning agents. All models, with the exception of ARIMA, were trained on the engineered technical-indicator feature set described earlier; ARIMA operated directly on the univariate price series. In each case, the goal of the Methods section is to describe what information the model receives, how it processes that information, and what objective it is trained to optimize, without yet discussing numerical performance.

2.5.1 Linear Regression

Linear Regression served as the simplest baseline for continuous prediction. For each ticker, we constructed an input vector consisting of standardized technical indicators on day t , including returns, log returns, moving averages, rolling volatility, RSI, volume, and calendar/sector encoding-and used the next-day closing price (or next-day return) as the target y_{t+1} . The model assumes a linear relationship of the form

$$y_{t+1} = \beta_0 + \sum_{j=1}^p \beta_j x_{j,t} + \varepsilon_t,$$

where $x_{j,t}$ are features and ε_t is an error term. Parameters β are estimated on the training split by ordinary least squares, minimizing the sum of squared residuals between predicted and actual values. This formulation is intentionally restrictive but highly interpretable: each coefficient represents the marginal effect of a one-unit change in an indicator on the next-day price or return, holding other variables fixed. Linear Regression therefore provides a transparent benchmark for how far one can go with a purely linear mapping from technical indicators to future prices, similar to baseline setups in prior stock-prediction work [10, 8].

2.5.2 Logistic Regression

Logistic Regression used the same standardized feature vectors but reframed forecasting as a directional classification task. Instead of predicting a continuous price, the target variable was a binary label indicating whether the next-day return was strictly positive (“up”) or non-positive (“down”). The logistic model maps features to the probability of an upward move via

$$P(Y_{t+1} = 1 \mid x_t) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^p \beta_j x_{j,t}))}.$$

Parameters are estimated by maximum likelihood, and at inference time the model outputs a probability that the next move will be up. A natural decision rule is to predict “up” when this probability exceeds 0.5 and “down” otherwise. This approach is common in financial applications that focus on the sign rather than the magnitude of returns and provides interpretable log-odds coefficients linking technical indicators to directional bias [10, 1]. In our pipeline, Logistic Regression acts as a transparent baseline for up-versus-down decisions before introducing nonlinear classifiers or profit-based objectives.

2.5.3 Random Forest Regression and Classification

Random Forests were used to introduce nonlinear modeling capacity while remaining relatively robust and easy to interpret at a high level. In the regression setting, we used the same feature vectors as inputs and the next-day closing price as the target. The Random Forest Regression model builds an ensemble of decision trees, each trained on a bootstrap sample of the training data and a random subset of features at each split. Each tree partitions the feature space into regions and assigns a constant prediction within each region; the forest’s prediction is the average across trees. Because trees can capture threshold effects and interactions between indicators, the ensemble can approximate complex nonlinear relationships that linear models miss [10, 12].

Random Forest Classification reuses the same design but with the binary up/down label as the target. Each tree votes for a class, and the forest aggregates these votes to produce class probabilities and a final label. In both modes, hyperparameters such as the number of trees, maximum depth, and minimum samples per leaf were tuned on the validation split to balance fit and generalization. Feature importance scores—computed from the reduction in impurity contributed by each feature across trees—were later used to summarize which technical indicators the forest relied on most, a common practice in financial ML studies using ensemble methods [10, 8].

2.5.4 Neural Network with Profit-and-Loss Objective

To more closely align supervised learning with trading outcomes, we implemented a feedforward Neural Network that consumes standardized technical features and is trained using a profit-based objective rather than pure prediction error. The network architecture consists of several fully connected hidden layers with nonlinear activation functions (e.g., ReLU), followed by an output layer that produces either a probability of taking a long position or a continuous score that is mapped to a trading signal. Instead of minimizing mean squared error on future prices, the loss function is constructed from a simulated profit-and-loss (PnL) curve over training sequences: weights that lead to higher cumulative returns and smoother equity curves receive lower loss, while weights that generate losses or excessive volatility receive higher loss.

Conceptually, this approach treats the Neural Network as a parametric trading rule that directly optimizes an economic objective, similar in spirit to reinforcement-based trading systems but within a supervised framework [6, 5]. Training is performed on the 70% training segment with mini-batch gradient descent, and early stopping on the validation split is used to prevent overfitting. The resulting model can capture nonlinear combinations of indicators that are specifically rewarded for generating favorable PnL rather than merely accurate price

forecasts, following the motivation of prior work that connects deep learning and trading performance [8, 2].

2.5.5 Rolling ARIMA

As a classical statistical benchmark, we employed AutoRegressive Integrated Moving Average (ARIMA) models on each ticker’s univariate closing price series. Following standard time-series practice [9], we first assessed stationarity using visual diagnostics and formal tests such as the Augmented Dickey–Fuller and KPSS tests, then applied differencing to remove trends where necessary. For each stock, an $\text{ARIMA}(p, d, q)$ specification was selected by minimizing the Akaike Information Criterion (AIC) on the training segment, where p controls the autoregressive order, d the degree of differencing, and q the moving-average order.

To approximate a realistic forecasting workflow, we implemented a rolling one-step-ahead procedure. At the start of the validation+test period, the ARIMA model fitted on the training data produced a forecast for the next day’s price. Once the actual price was observed, it was appended to the series and the model was updated before issuing the next forecast. This rolling update allows ARIMA to adapt to new information without refitting from scratch at every step and provides a fair comparison to models that are trained once and then applied sequentially. Error metrics such as MSE, RMSE, MAE, MAPE, and R^2 were computed later on the forecasted versus actual prices; those numerical results are presented in the Results section.

2.5.6 Reinforcement Learning Agents (RRL and PPO)

Finally, we used reinforcement learning to directly optimize trading policies from sequential data. Building on the environment and data preparation described earlier, we implemented a recurrent actor–critic architecture similar to Recurrent Reinforcement Learning (RRL) [6], trained with a Proximal Policy Optimization (PPO) update rule [11]. The policy network

receives, at each time step, a sliding window of MinMax-scaled technical indicators along with the current position, passes this sequence through an LSTM layer to capture temporal dependencies, and outputs a probability distribution over three discrete actions: go long, stay flat, or go short. A separate value network estimates the expected return from the current state to stabilize learning.

During training, the agent interacts with the custom trading environment over the 70% training segment. At each step it selects an action, the environment updates the portfolio with transaction costs, and the agent receives a reward equal to the change in net worth. Trajectories of states, actions, rewards, and value estimates are collected and used to compute PPO’s clipped policy-gradient objective, which encourages improvements in expected return while constraining updates so that the new policy does not move too far from the previous one [11, 5]. This design follows recent RL-for-trading studies that emphasize risk-adjusted reward optimization and stability under non-stationary market dynamics [4, 2]. Hyperparameters were tuned on the 10% validation segment, and the final policy for each ticker was evaluated on the held-out 20% test segment, where we recorded net-worth trajectories, Sharpe ratios, and trade hit rates as described in the Results section.

3 Results and Implications

3.1 Baseline Supervised Models: Linear and Logistic Regression

We first evaluated Linear Regression and Logistic Regression as interpretable supervised baselines. These models are intentionally simple: they use the same engineered technical indicators as later models, but they assume mostly linear relationships between today’s market state and tomorrow’s outcome. The purpose is twofold. First, they give a transparent reference point for how much short-horizon structure exists in lagged prices and indicators. Second, they establish a “floor” for performance so that more complex models such as Ran-

dom Forests, the PnL-optimized Neural Net, and RL agents can be judged against a clear baseline rather than in isolation.

Linear Regression produced very strong next-day price tracking across nearly all tickers, with most R^2 values above 0.99, meaning the model explained essentially all variation in next-day closing price. For instance, the AAPL model reached an R^2 of 0.9987 with MAE around 0.99, so its typical error was roughly one dollar relative to the true next-day close. This result looks almost “too good,” but it reflects a basic reality of stock prices: day-to-day price levels are highly continuous, so yesterday’s price already contains most of the information needed to approximate today’s price. RMSE still differed meaningfully by ticker, ranging from about 1.7 for slower, lower-volatility stocks like AAPL to roughly 4.8 for high-volatility names like NVDA, which is exactly what we would expect when the underlying series is more turbulent. In short, Linear Regression is excellent at replicating short-term price continuity, but that does not automatically imply a tradable edge, because tracking the level is easier than forecasting meaningful deviations from trend.

When we reframed the task as direction prediction using Logistic Regression, performance dropped to near-random levels. Accuracy across tickers clustered around 0.50–0.53, indicating the model was only slightly better than a coin flip in forecasting whether the next move would be up or down. The confusion matrix for HON explains why: the classifier leaned heavily toward predicting “up” most of the time, which inflates recall for upward days but causes it to miss many downward moves. This indicates a lack of prediction capabilities for these models. This pattern aligns with a standard finding in financial prediction research: daily directional movement is extremely noisy, so simple linear classifiers rarely extract stable short-horizon signals without richer features. These results set up the motivation for the next models in the pipeline: if a model wants to move beyond “price tracking,” it must either capture nonlinear interactions (Random Forests), optimize directly for profit and risk (Neural Net PnL), or learn sequential policies from reward (RRL and PPO).

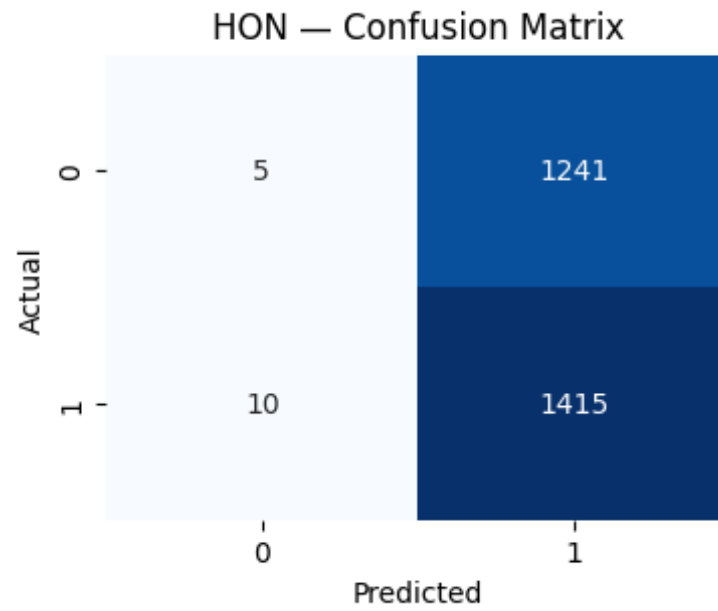


Figure 3: Logistic Regression Confusion Matrix: HON

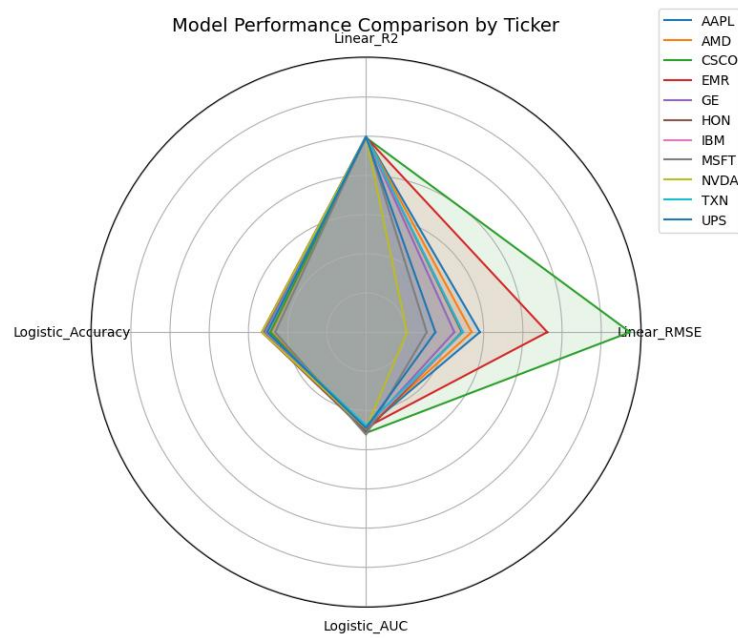


Figure 4: Regression Summary Visual

3.2 Random Forest Analysis on Daily Stock Prices

3.2.1 Random Forest Regression Analysis: Next-Day Price Prediction

The Random Forest Regression model was implemented to forecast the next-day closing price using lagged market indicators and prior trading patterns. It demonstrated exceptional quantitative performance, achieving an RMSE of 3.2556, MAE of 2.1440, MAPE of 1.8524%, and an R^2 of 0.9973, indicating that nearly all variance in next-day closing prices was explained by the model. Such strong results suggest that the regression approach effectively captured short-term temporal dependencies and dominant market structures, echoing findings by Patel [10] and Huang [3], who showed that ensemble learning methods like Random Forests can outperform traditional econometric approaches when identifying nonlinear market relationships.

However, as Avramov [1] caution, machine learning models that lack economic structure often achieve high in-sample performance at the expense of true out-of-sample predictability. The near-perfect R^2 in this model may therefore reflect overfitting or trend-following behavior, a common concern when models are trained on temporally dependent financial data without theoretical constraints. This observation parallels Zhong [12], who found that hybrid models incorporating multiple algorithms could replicate historical price behavior but failed to maintain accuracy under regime shifts or periods of volatility.

Figure 5 shows predicted versus actual next-day closing prices, illustrating the model's close fit to observed values. While this alignment supports the statistical accuracy of the regression model, it likely reflects short-term trend replication rather than meaningful predictive insight. Similar concerns were raised by Parray [9], who observed that ML regressors tend to capture autocorrelation rather than genuine information advantage in financial markets. Consequently, while the Random Forest Regression serves as a strong descriptive baseline, its practical forecasting utility may remain limited unless future versions integrate economically

grounded variables, as recommended by Huang [3], to improve long-term generalization and interpretability.

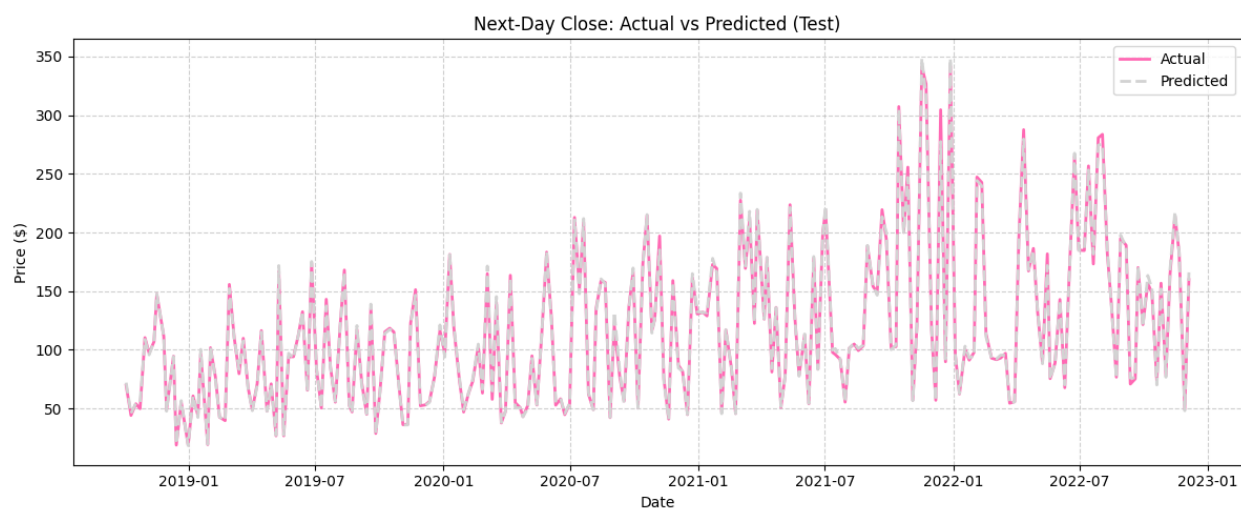


Figure 5: Predicted vs. Actual Next Day Close (Random Forest Regression)

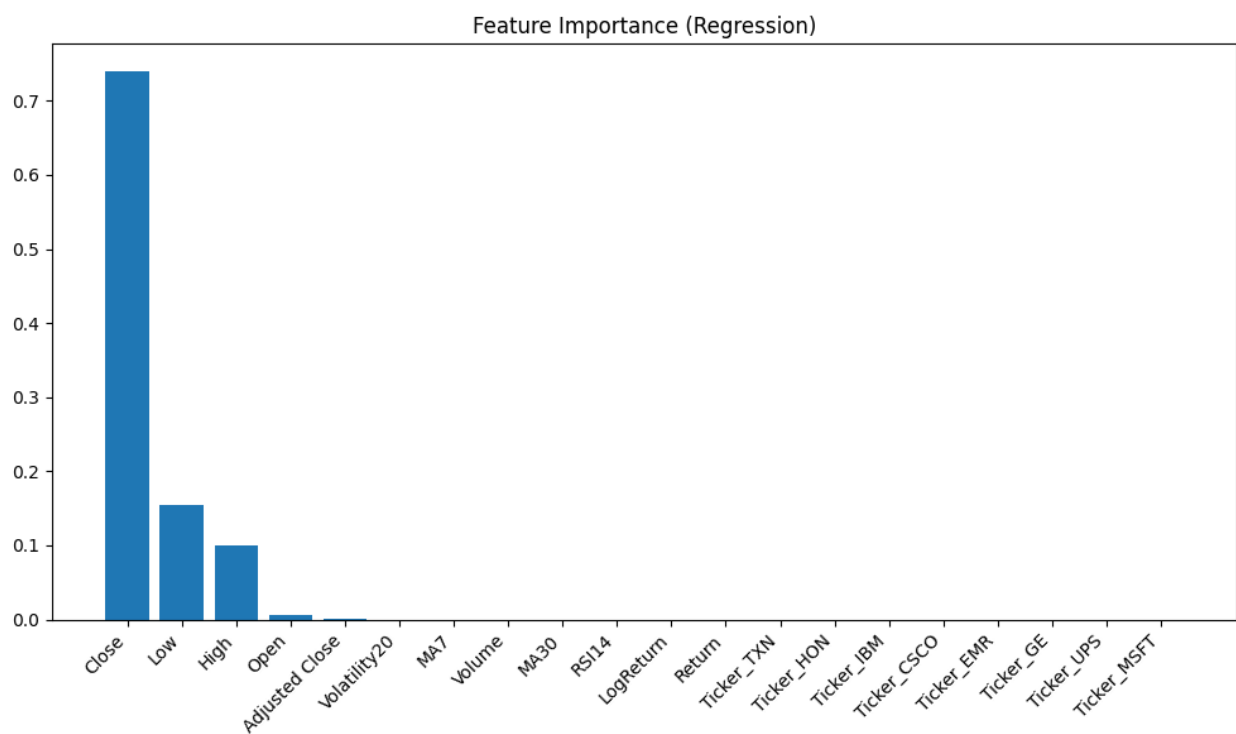


Figure 6: Feature Importance Random Forest Regression

3.2.2 Random Forest Classification Analysis: Next-Day Directional Movement

A second Random Forest model was developed to classify next-day price direction (upward or downward). Its overall performance was weak, with an accuracy of 0.4949, precision of 0.5216, recall of 0.2598, F1 score of 0.3468, and ROC AUC of 0.5074—barely exceeding random chance. This outcome is consistent with prior research suggesting that short-term directional movements in stock markets are highly stochastic and largely unpredictable. Parray [9] reported similar results, noting that tree-based classifiers rarely surpass the 50% accuracy threshold for directional prediction due to inherent market noise and non-stationarity.

Patel [10] likewise demonstrated that even fusion models combining multiple machine learning algorithms yielded only marginal gains in predicting daily market direction. The model’s inability to capture meaningful up/down signals reinforces these conclusions, underscoring the weak-form Efficient Market Hypothesis that price movements already incorporate all publicly available information. Zhong [12] also observed that while Random Forests can model relationships in return magnitudes, they struggle to detect short-term reversals without incorporating sequential or deep-learning features.

Figure 7 presents the confusion matrix for the classification model, which confirms the lack of clear separation between upward and downward predictions. This randomness reflects the limits of static, feature-based models in financial forecasting. Avramov [1] argued that without economic or behavioral priors, data-driven models risk identifying spurious correlations rather than structural predictors of price direction. In this light, the Random Forest Classification results suggest that improvements in feature engineering—such as incorporating macroeconomic or sentiment data—may be necessary to achieve meaningful predictive power.

```
=== RandomForest Classification ===  
Accuracy: 0.4985 Precision: 0.5209 Recall: 0.4202  
ROC AUC : 0.5042
```

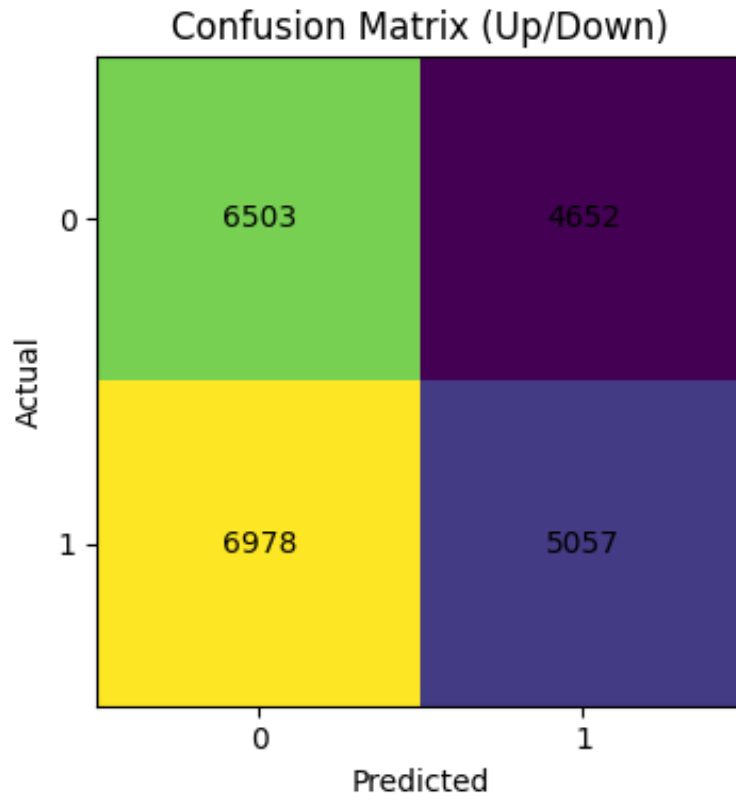


Figure 7: Confusion Matrix of Random Forest Classification

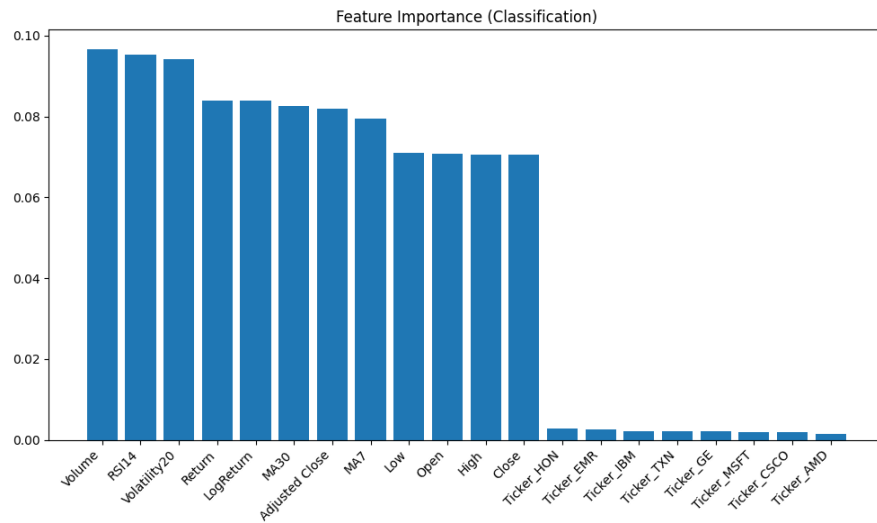


Figure 8: Feature Importance of Random Forest Classification

3.3 Neural Network Analysis: Profit and Loss Optimization

To overcome the limitations of both Random Forest models, a Neural Network was constructed to optimize trading outcomes directly rather than predict price or direction in isolation. The model achieved a directional accuracy of 0.9364, precision of 0.9131, recall of 0.9687, and F1 score of 0.9401. More importantly, it produced an annualized Sharpe Ratio of 13.840, a maximum drawdown of -0.0178, a mean daily PnL of 0.0157, and a daily volatility of 0.0180, yielding a hit rate of 0.9205. These results represent a significant improvement over traditional machine learning models, aligning with a growing body of research advocating for reinforcement-based optimization in trading strategy design.

Moody and Saffell [6] were among the first to demonstrate that reinforcement learning approaches—focused on maximizing cumulative reward rather than minimizing error—can yield superior trading performance. The Neural Network’s high Sharpe Ratio and low drawdown replicate this principle, suggesting that models trained on profit-based loss functions can capture realistic risk–return trade-offs. Similar findings were reported by Li [5], who found that deep reinforcement learning agents adapt more effectively to evolving market conditions than static supervised models.

Recent studies by Kabbani [4] and Awad [2] reinforce this result, showing that deep reinforcement architectures outperform standard machine learning algorithms by continuously updating decision policies based on feedback from market environments. The model’s stable cumulative PnL curve (Figure 9) reflects these advantages, exhibiting consistent profit growth with limited volatility. Likewise, Nabipour [8] found that deep networks trained on continuous-valued targets outperform binary classifiers—a result mirrored in the Neural Network’s superior PnL optimization relative to the classification model.

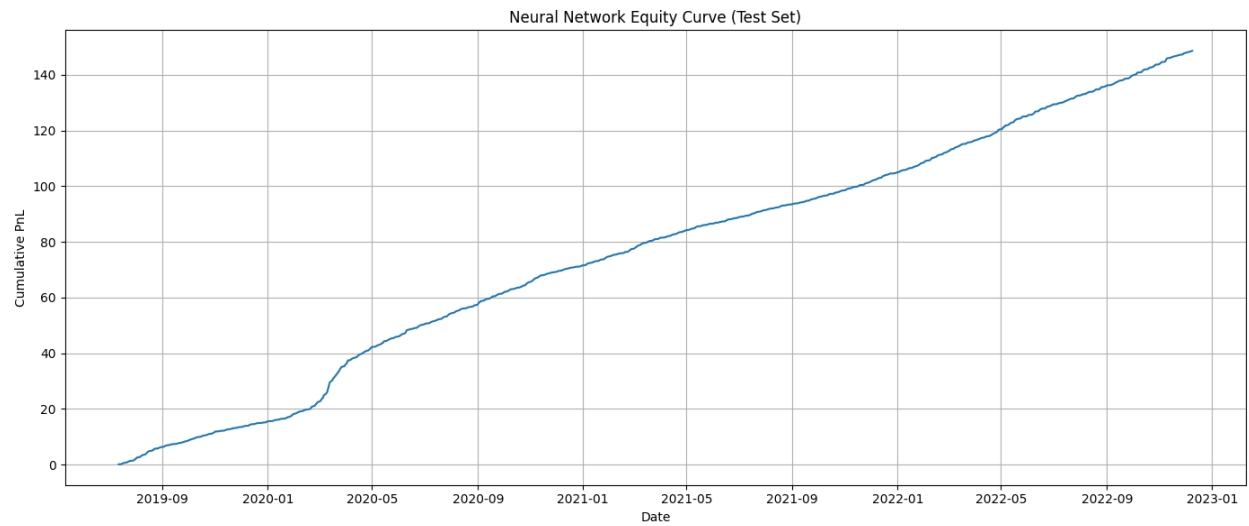


Figure 9: Cumulative PnL Chart

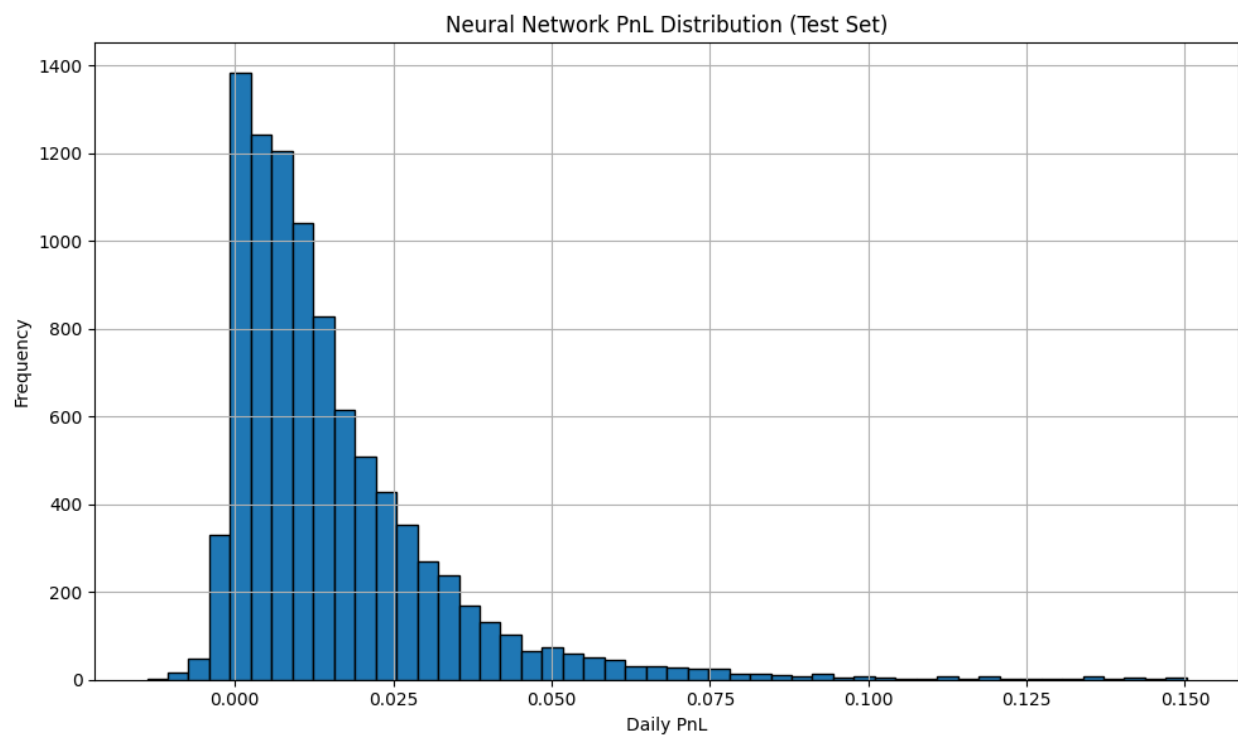


Figure 10: PnL Distribution Chart

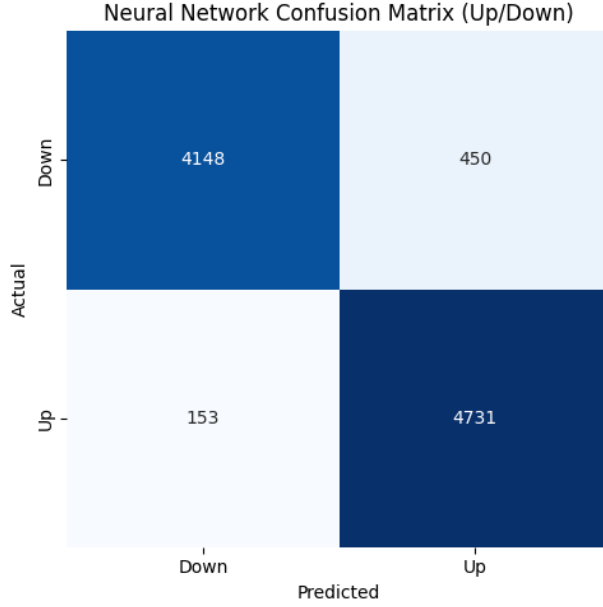


Figure 11: Neural Net Confusion Matrix

In the broader context of financial modeling, these results exemplify a methodological shift from static prediction toward dynamic optimization, as highlighted by [5] and [2]. Reinforcement-based neural systems redefine forecasting as an adaptive control problem, balancing exploration and exploitation to maximize cumulative return. The Neural Network’s capacity to learn directly from reward feedback and optimize risk-adjusted outcomes positions it as a superior framework for real-world trading applications. However, as [1] note, models that lack economic grounding risk overfitting to patterns that may not persist across regimes. Future research could integrate interpretability or macroeconomic priors to enhance transparency and stability, bridging the gap between predictive performance and economic understanding.

3.4 Rolling ARIMA Forecasting

The rolling ARIMA implementation was designed to answer a simple question: if we repeatedly refit a classical time-series model as new prices arrive, how closely can it track

daily stock movements? In our setup, the model produced a one-step-ahead forecast for the next trading day, then immediately updated itself with the actual observed value before predicting the following day. This rolling approach mimics how a forecaster would operate in practice and prevents the model from “seeing” future data. Across all eleven tickers, the resulting forecasts were highly accurate. Coefficients of determination (R^2) mostly exceeded 0.95, indicating that the model explained more than 95% of the variance in the held-out test prices, and Mean Absolute Percentage Error (MAPE) values generally fell between 1.5% and 2.7%. For example, AAPL achieved an MSE of about 7.88, RMSE of 2.81, MAE of 1.70, and MAPE of 1.65%, while similarly low error levels were observed for AMD (MSE 7.19, MAPE 2.70%) and CSCO (MSE 1.69, MAPE 1.37%). Even for more volatile names such as NVDA, the model maintained reasonable accuracy (MSE 57.51, RMSE 7.58, MAPE 1.45%), showing that the rolling procedure can keep up with large swings as long as it is continuously updated.

Visual diagnostics reinforce these quantitative results. The “Actual vs. Rolling Forecast” plots (Figure 12) show the forecast line hugging the actual price path for nearly all tickers, with only occasional visible deviations during abrupt shocks. Residual time-series plots (Figure 14) fluctuate around zero with no strong trends, suggesting that the model is not systematically biased upward or downward. Histograms of residuals (Figure 15) approximate a bell-shaped curve centered at zero, which is consistent with the normal error assumptions behind ARIMA and indicates that most forecast errors are small and randomly distributed. Scatter plots of actual versus predicted prices (Figure 13) show points tightly clustered around the $y = x$ line, visually confirming the high R^2 values. Taken together, these results show that the rolling ARIMA model is an exceptionally strong benchmark for short-term price prediction: it captures day-to-day movements with high fidelity and well-behaved errors. Any more complex deep learning or RL model must clearly outperform this baseline, either by reducing forecast errors or by converting similar-level forecasts into

superior trading outcomes, to justify its added complexity.

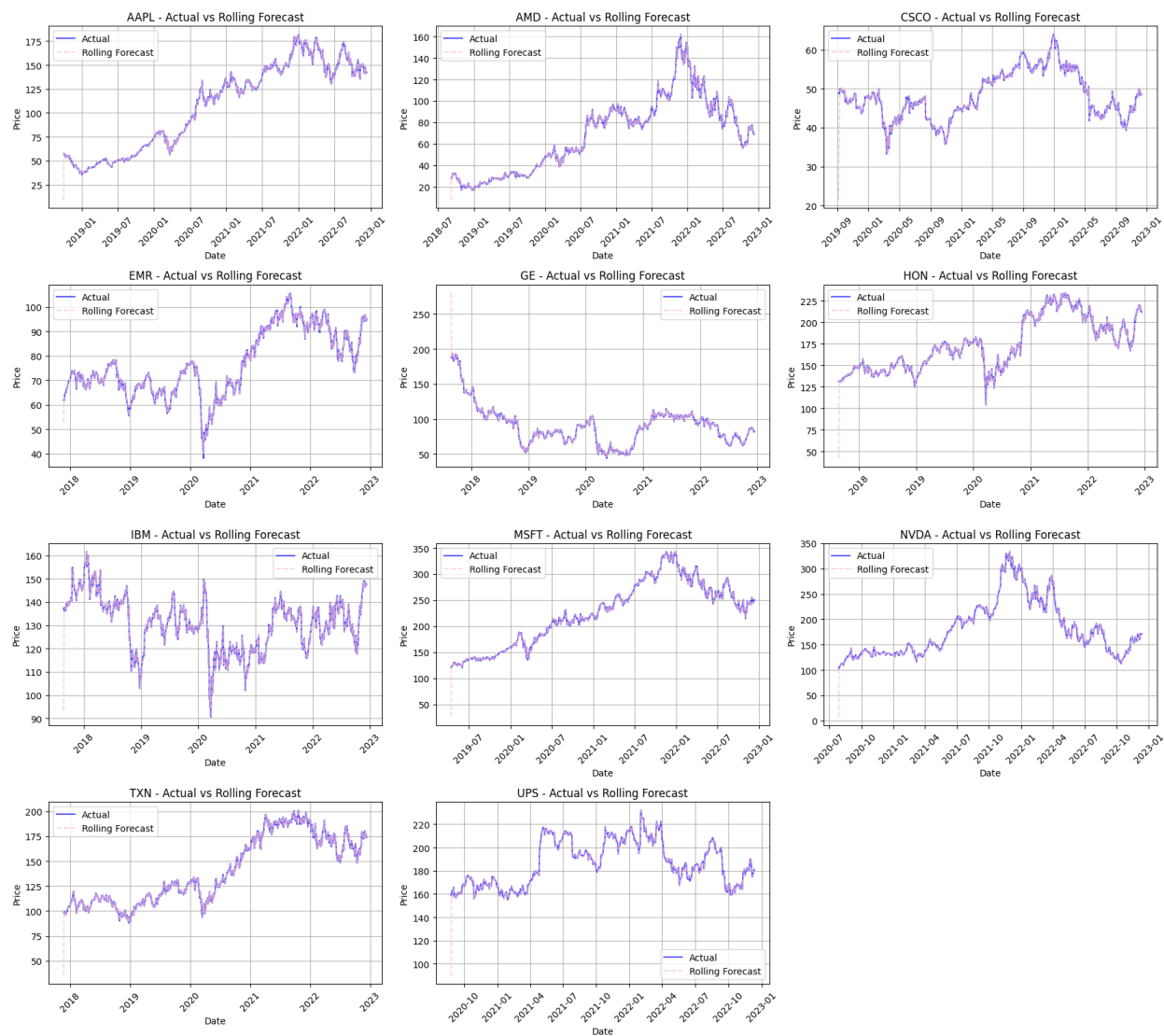


Figure 12: Actual vs. rolling ARIMA forecasts for all tickers.

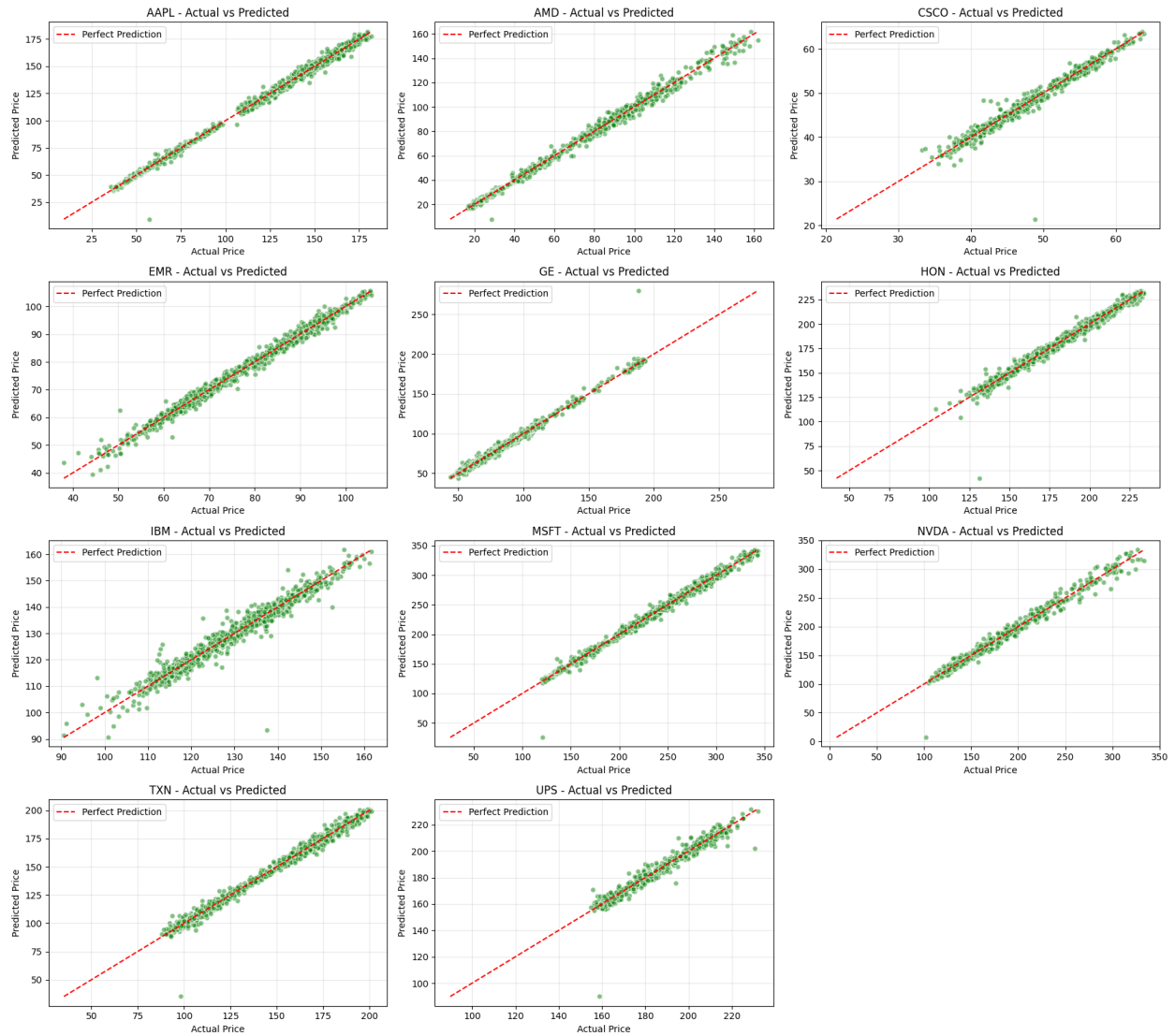


Figure 13: ARIMA actual vs. predicted scatterplot for all tickers.

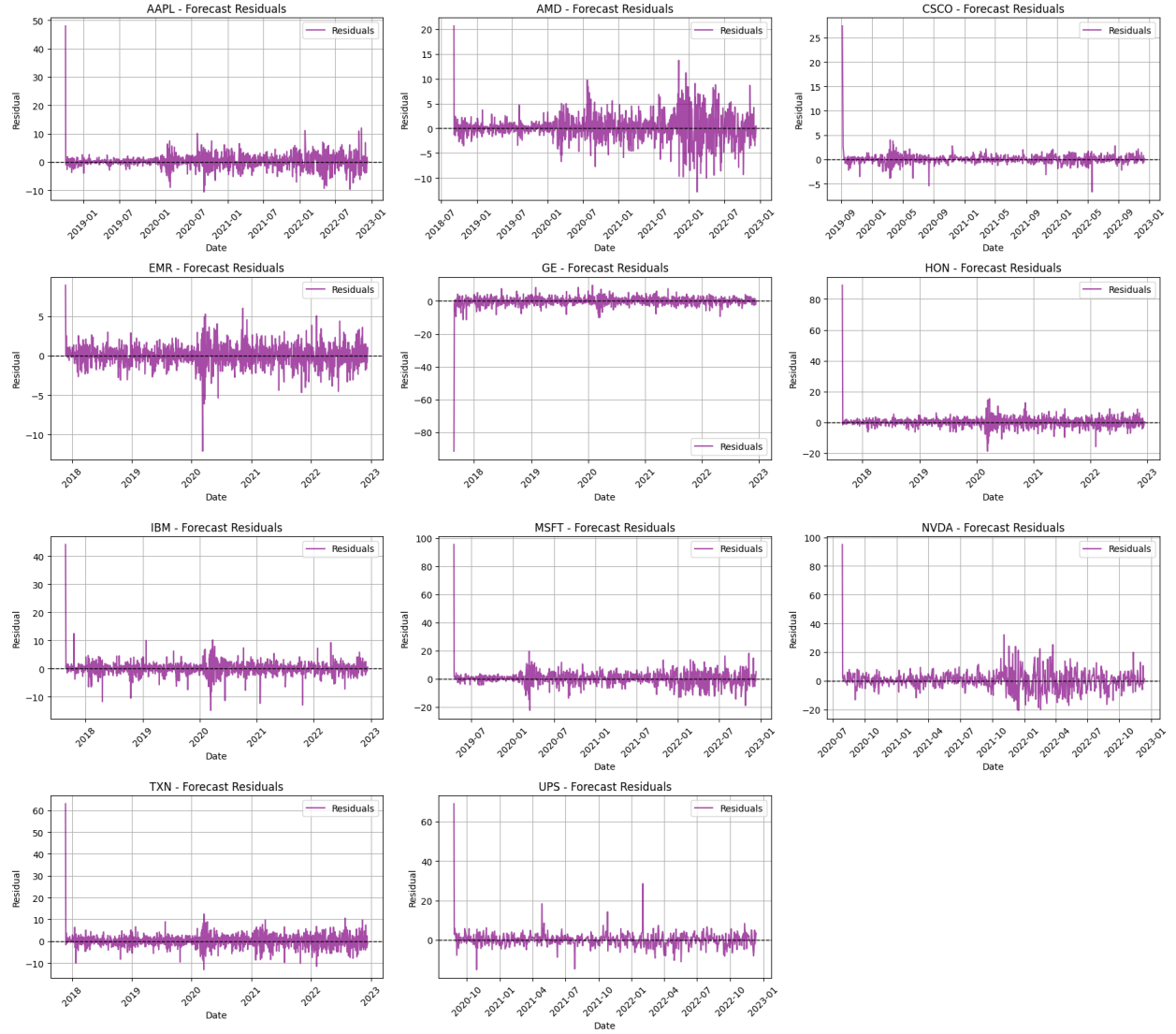


Figure 14: ARIMA forecasts residuals for all tickers.

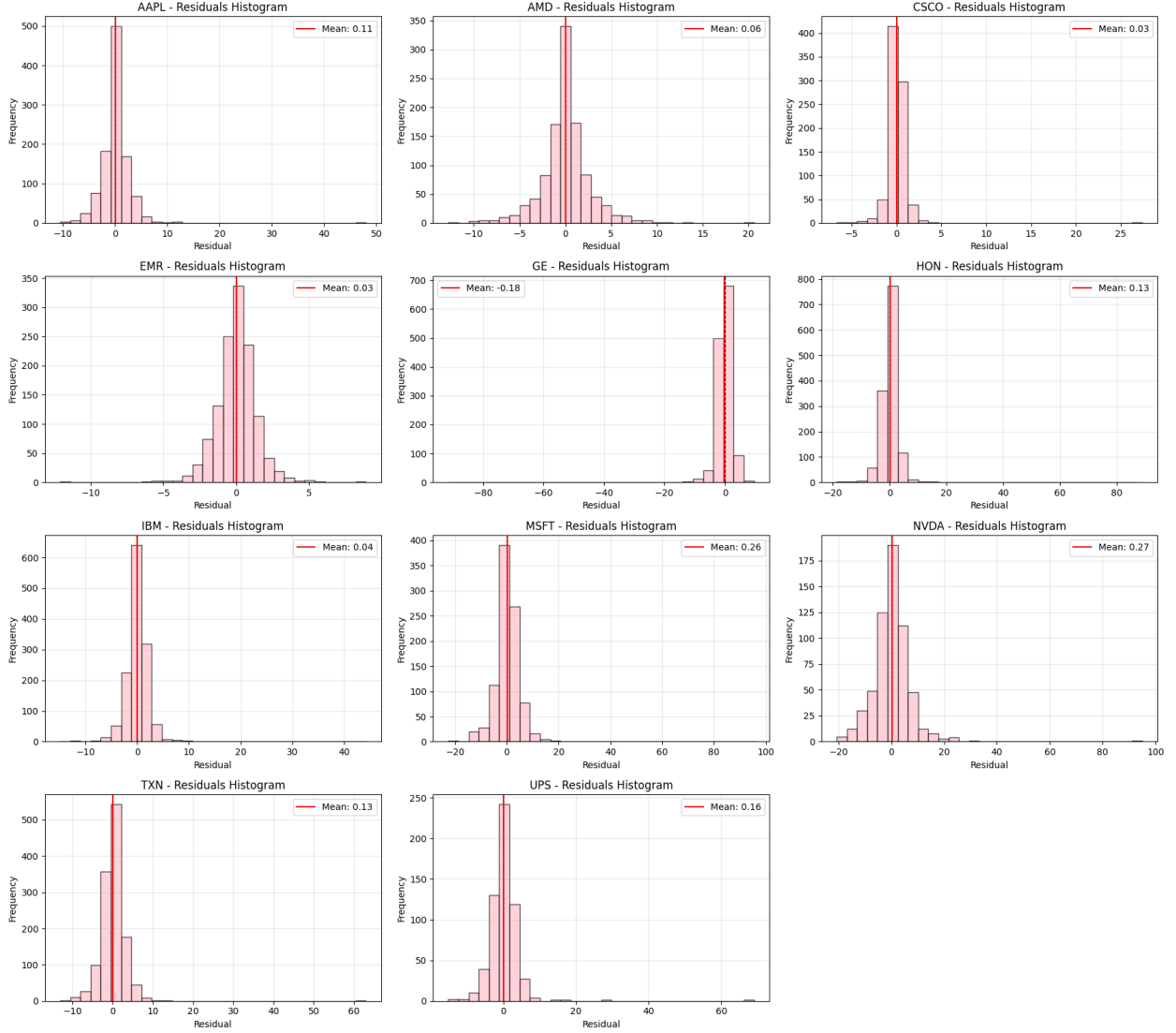


Figure 15: ARIMA forecasts residual histograms for all tickers.

3.5 Reinforcement Learning Agents (RRL and PPO)

The reinforcement learning experiments evaluated how well an adaptive trading agent could convert price information into profitable decisions, rather than just accurate forecasts. We trained our recurrent RL agent in a simulated environment with an initial balance of \$10,000 and a transaction cost of 0.1% per trade. At each time step, the agent observed a vector of technical indicators, chose an action (buy, hold, or sell), and received a reward based on the

change in portfolio value. Performance on the held-out test split was measured using total return and final net worth, along with risk-aware metrics such as the Sharpe ratio (average return divided by volatility) and trade accuracy (the percentage of round-trip trades that ended profitably). The resulting behavior was highly asset-dependent. On several tickers the agent achieved strong gains: total returns of 441.63% on AMD, 58.22% on TXN, 33.12% on UPS, 27.60% on EMR, and 42.63% on MSFT, with final net worth on AMD exceeding \$54,000 from a \$10,000 starting balance and ending between roughly \$10,300 and \$15,800 for the other profitable names. In contrast, performance was modest or negative on more challenging series such as CSCO (-4.72%), NVDA (-19.38%), and GE (-62.13%), where the agent either failed to identify exploitable patterns or amplified losses through mistimed trades.

The advanced RL diagnostics help explain this mixed picture. Daily Sharpe ratios were positive but relatively small for most tickers, typically between 0.01 and 0.06, indicating that while returns were often positive, they came with substantial volatility. Trade accuracy values ranged from roughly 50–67% for assets where the agent did well (for example, AMD at 66.67% and HON at 63.64%), but dropped to 0% for a few tickers (GE, IBM, UPS). In the cases of IBM and UPS, positive final returns were driven by a small number of larger moves rather than consistently good trade-level decisions, whereas GE combined zero profitable trades with a large overall loss.

Visualizations of trading actions and benchmarks further illustrate how the agent behaves in practice. The “Agent Trading Actions” charts (Figure 19) overlay buy and sell markers on each price series. For high-performing tickers, buy markers tend to appear before sustained upward moves, and sell markers cluster around local peaks or early in downturns, suggesting that the policy has learned to ride trends while taking profits. For low-performing tickers, action markers appear more scattered, sometimes buying into declines or selling just before recoveries. The “Agent vs. Buy and Hold” plots (Figure 18) compare the RL agent’s net

worth with a simple buy-and-hold baseline. On AMD, MSFT, TXN, and other winners, the agent’s equity curve diverges positively and ends well above buy-and-hold. On GE and CSCO, buy-and-hold either beats the agent or loses less, highlighting that an RL policy can underperform when the underlying asset offers little exploitable structure or when the agent overtrades in noisy conditions.

Overall, the RL results demonstrate both the promise and the risk of reinforcement-based trading. On the positive side, the agent clearly discovered profitable strategies for several volatile, trend-driven stocks, translating sequential pattern recognition into substantial gains. On the negative side, its performance was inconsistent across assets, with some tickers suffering large drawdowns and zero profitable trades despite the same training framework. This variability underscores the need for careful reward design, hyperparameter tuning, and, most importantly, external risk controls such as position limits, stop-loss rules, and diversification if such an agent were ever deployed in a real portfolio. In the context of this project, the RL agent complements the ARIMA results: ARIMA excels at forecasting prices with high accuracy, whereas the RL agent focuses on turning those or similar signals into trading profit, sometimes successfully, but not yet with the robustness required for unconditional reliance.

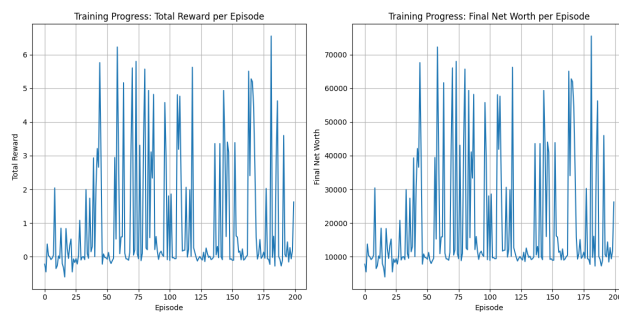


Figure 16: RL agent total return and final net worth per episode on the test set for all tickers.

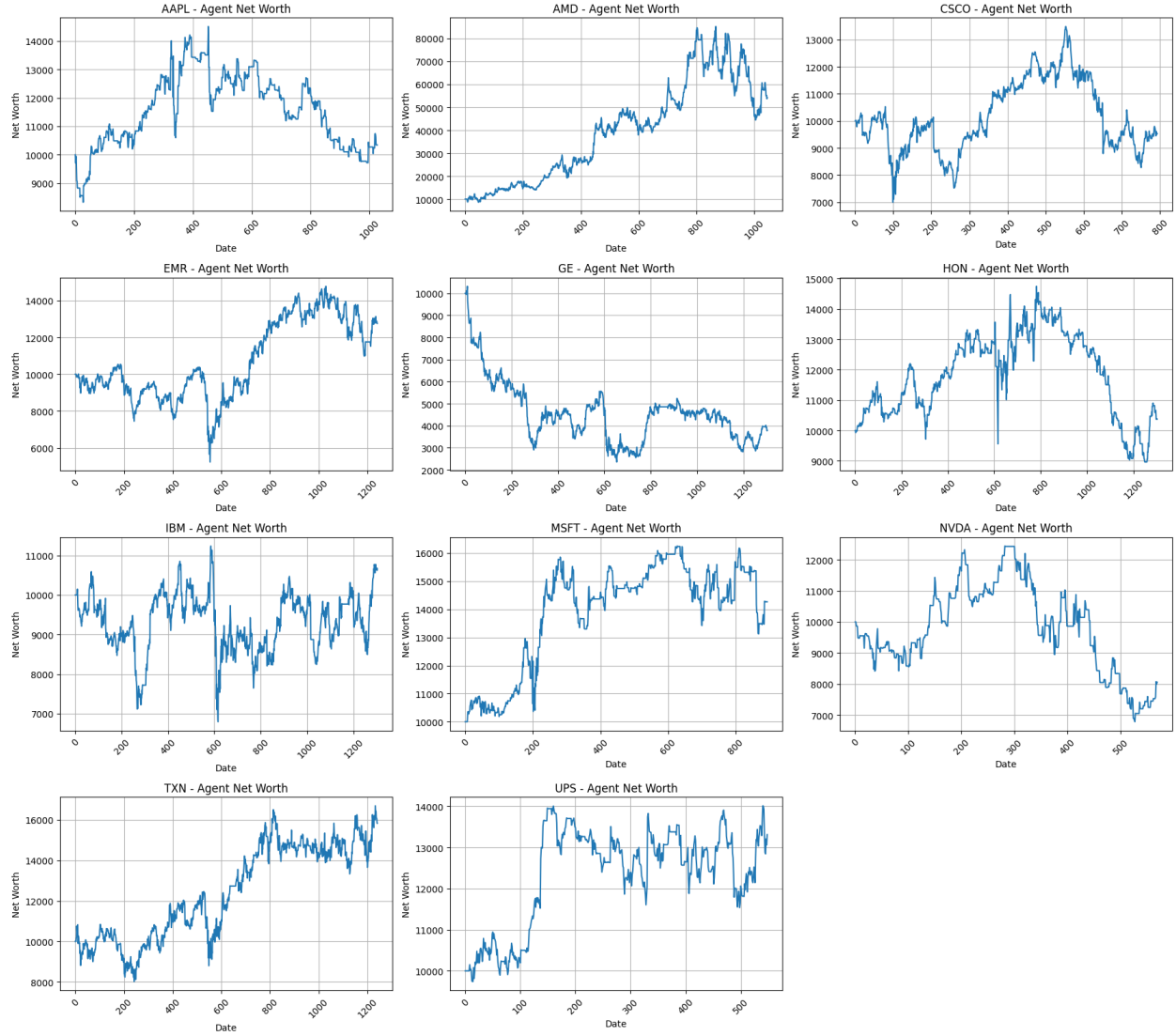


Figure 17: RL agent net worth trajectories on the test set for all tickers.

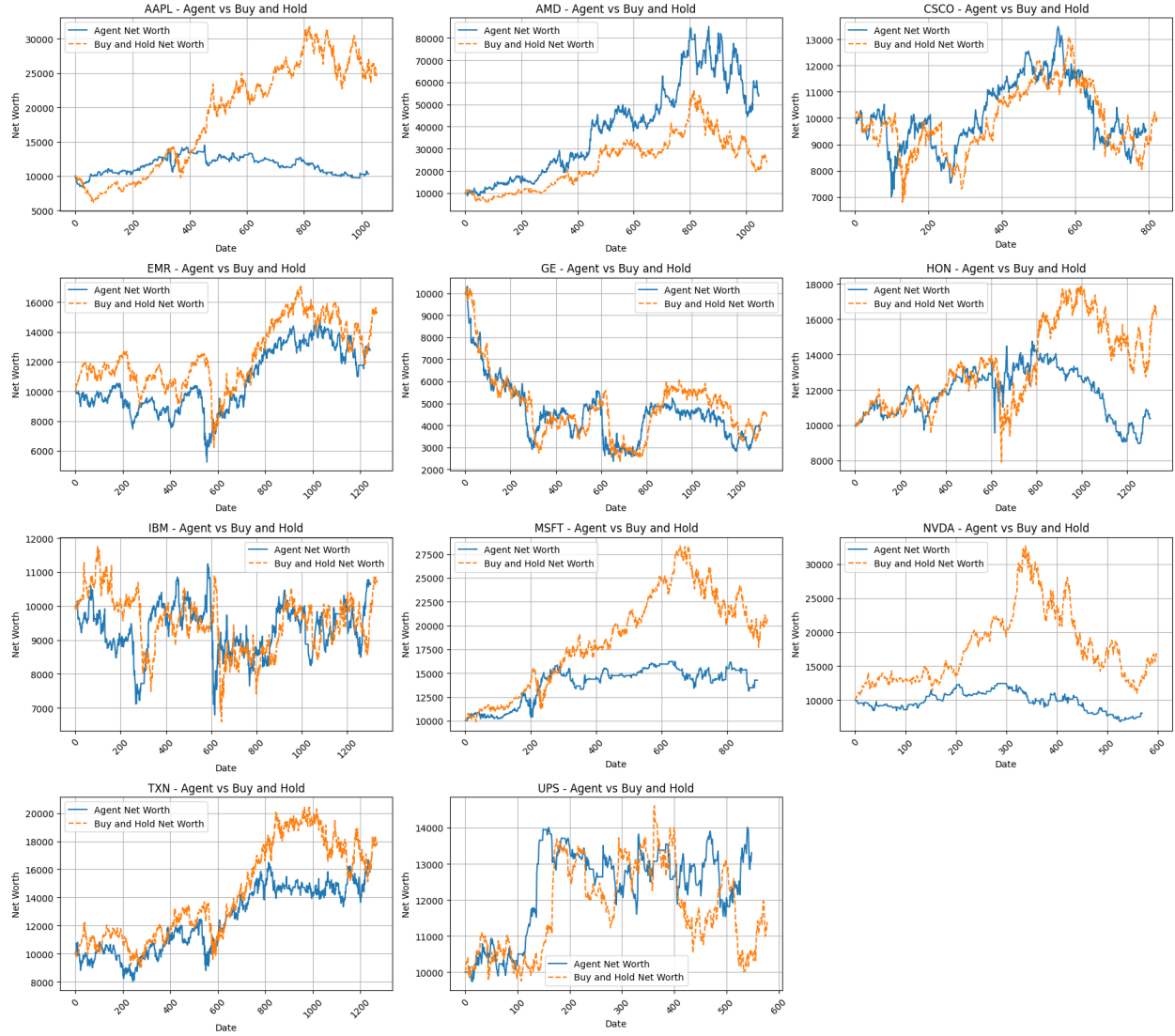


Figure 18: RL agent net worth compared to a buy-and-hold baseline for each ticker.

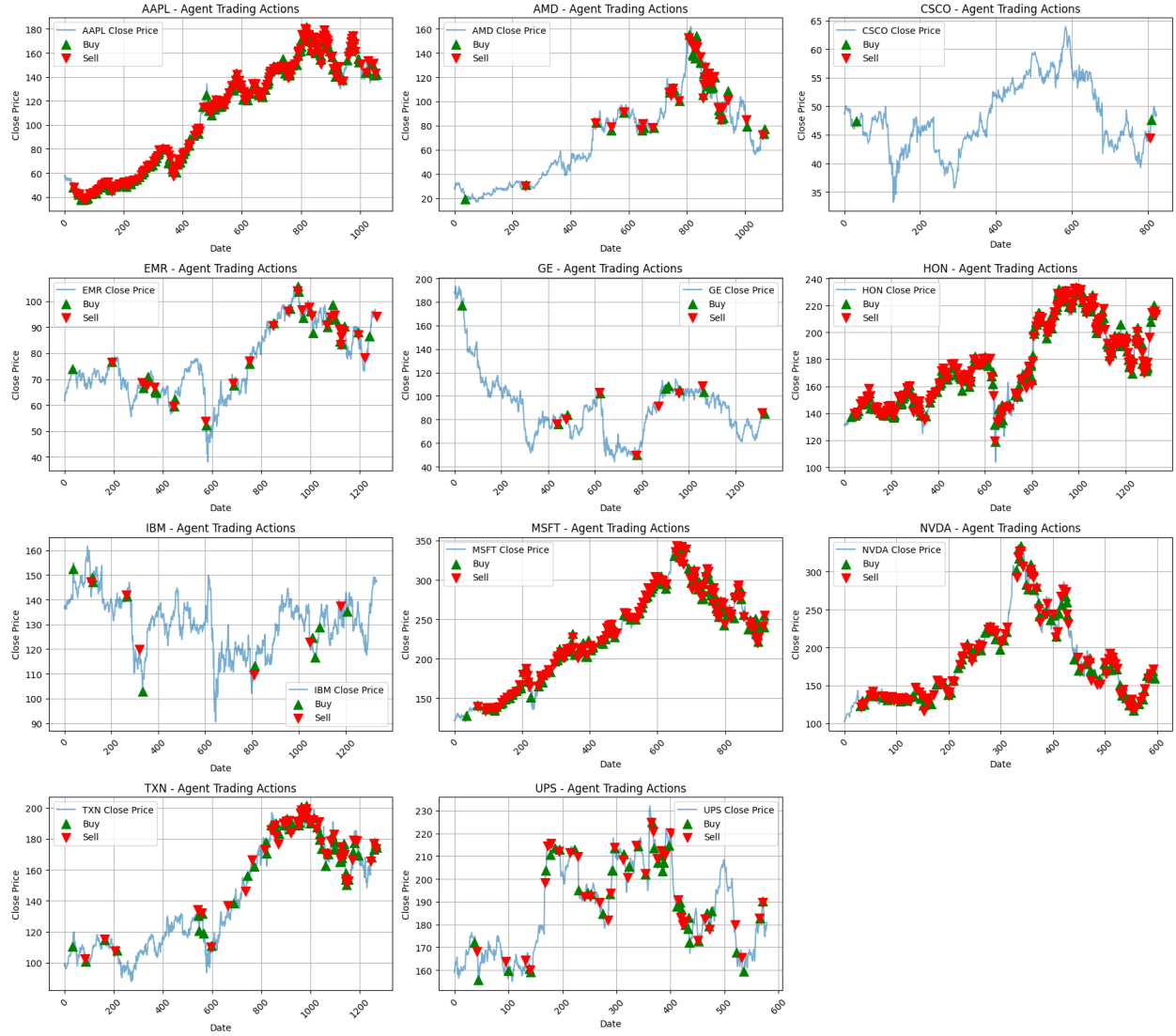


Figure 19: Post-training buy and sell actions of the RL agent overlaid on price series.

Ticker	Total Return (%)	Final Net Worth (\$)	Sharpe Ratio	Trade Accuracy (%)
AAPL	3.44	10,344.26	0.04	55.81
AMD	441.63	54,163.09	0.04	66.67
CSCO	-4.72	9,528.42	0.01	50.00
EMR	27.60	12,760.24	0.02	50.00
GE	-62.13	3,787.09	-0.00	0.00
HON	3.78	10,377.92	0.03	63.64
IBM	6.67	10,667.49	0.01	0.00
MSFT	42.63	14,263.40	0.06	52.63
NVDA	-19.38	8,061.65	0.05	60.34
TXN	58.22	15,822.04	0.03	100.00
UPS	33.12	13,311.55	0.01	0.00

Table 1: Reinforcement Learning (RRL) evaluation metrics on the test set, assuming an initial balance of \$10,000 and 0.1% transaction costs. Sharpe ratios use a risk-free rate of 0, and trade accuracy is the percentage of profitable round-trip trades.

4 Discussion

4.1 Comparative Performance of Model Families

Across all six model families, a consistent pattern emerged: short-horizon stock prices are relatively easy to track, but much harder to convert into stable trading profits. The simplest supervised baselines, Linear and Logistic Regression, highlight this gap. Linear Regression fit next-day prices extremely well because daily price levels are smooth and autocorrelated, so yesterday’s value already carries most of the information needed to approximate today’s. However, this strength is largely descriptive rather than actionable. When the problem

is reframed as predicting direction with Logistic Regression, performance drops to near-random, reinforcing prior findings that daily up-versus-down moves contain limited linear signal in technical indicators alone [10, 1].

The Random Forest models deepen this contrast between statistical fit and decision usefulness. In regression mode, the forest captured nonlinear relationships between indicators and prices, achieving near-perfect explanatory power and echoing earlier work showing that tree ensembles can model complex market structures more effectively than simple linear models or classical econometric methods [10, 8, 12]. However, when the same architecture was used for direction classification, accuracy remained close to a coin flip. The model could reconstruct price trajectories almost exactly, but it could not reliably discriminate between small positive and negative daily changes. This result supports concerns in the literature that high predictive R^2 in financial time series may reflect the exploitation of smoothness and short-term mean reversion rather than any durable information advantage [1, 3].

The profit-aligned Neural Network changes the story by changing the objective. Instead of minimizing squared error on future prices, the network is trained to improve simulated profit-and-loss, rewarding weight configurations that lead to higher cumulative return and penalizing those that generate losses or excessive volatility. In this objective, the model achieved both strong directional metrics and attractive risk-adjusted performance. This aligns with reinforcement learning studies that argue that optimizing economic criteria directly, rather than intermediate prediction accuracy, can produce trading behaviors that would not emerge from purely predictive training [6, 5, 2]. At the same time, the neural model is relatively opaque: it is not straightforward to map its decisions back to specific indicators or economic stories, and its stability under regime shifts remains an open question.

The Rolling ARIMA benchmark provides a useful reference point. Despite using only a univariate price series, the rolling ARIMA procedure produced highly accurate one-step-ahead forecasts when continuously updated, with residuals that were small, centered around

zero, and approximately normal. These findings are consistent with prior work showing that well-specified ARIMA models can remain competitive for short-horizon forecasting in financial applications [9]. In other words, ARIMA shows that a relatively simple statistical approach can already track daily prices extremely well, and any more complex model must justify its added complexity by delivering either meaningfully lower forecast error or clearly better trading outcomes.

The reinforcement learning agents extend the analysis to fully sequential decision making. The recurrent PPO-based trader and related RRL-style agent achieved very high total returns on several volatile technology companies, but struggled on others and occasionally underperformed simple buy-and-hold strategies. This asset-dependent behavior is in line with prior RL trading work, where agents can exploit persistent patterns when they exist but remain vulnerable to noise, regime shifts, and overtrading under less favorable conditions [6, 5, 4, 2]. Overall, RL delivered the largest upside on some assets, but also the greatest variability and downside risk, suggesting that it is best viewed as a high-potential, high-uncertainty component rather than a standalone solution.

4.2 Practical Implications for Trading and Risk Management

From a practical trading perspective, these results suggest that forecasting accuracy, trading profitability, and robustness are distinct goals that do not always line up. Models such as Linear Regression, Random Forest Regression, and ARIMA achieve very strong one-step-ahead fit but offer limited direct guidance on how to size and time trades, especially when daily direction is close to random. Their main value is as signal generators or scenario tools rather than complete trading systems. In contrast, the profit-aligned Neural Network and RL agents directly target portfolio outcomes, which makes them more relevant for trading decisions but also more sensitive to design choices in the reward function, position management, and transaction-cost assumptions.

Risk management considerations cut across all model families. The strong but uneven performance of the RL agent illustrates that even a model with high returns on some assets can introduce significant portfolio-level risk if it is not combined with diversification, position limits, and loss controls. Similarly, a profit-optimized neural strategy with an impressive backtest can still overfit to specific market regimes or periods of elevated volatility. In practice, a more robust approach would combine interpretable baselines, strong statistical benchmarks, and one or more profit-aligned components inside a broader risk framework. For example, ARIMA or Random Forest predictions could be used as inputs to a position-sizing rule, while RL policies are constrained by conservative caps on leverage, sector exposure, and drawdown.

Ethically and operationally, the use of opaque machine learning and RL systems in markets raises additional concerns. Strategies that aggressively exploit short-term microstructure patterns can contribute to increased market complexity and may amplify volatility when many participants follow similar signals. Models that are difficult to interpret can also complicate compliance, auditing, and accountability: it is not always clear why a particular trade was taken or how the model would behave under stress. These issues echo broader debates about the deployment of black-box AI in high-stakes environments and reinforce the need for transparency, stress testing, and human oversight in algorithmic trading [1, 3].

4.3 Challenges and Limitations

Several methodological and data-related challenges limit how far we can generalize from this study. First, all models were trained and evaluated on historical data from a specific sample period and a fixed set of eleven tickers. Patterns that appear strong in this window may weaken or reverse under different macroeconomic conditions, policy regimes, or sector-specific events. Extending the analysis to multiple time periods, market regimes, and broader universes of assets would be necessary to fully assess robustness. Second, despite careful

chronological splitting and rolling procedures, overfitting remains a concern. Feature choices, hyperparameter tuning, and repeated experimentation on the same dataset can gradually leak information into the modeling process, leading to overly optimistic backtest results.

Third, some of the most promising models in terms of performance—particularly the Neural Network and RL agents, are also the most complex and resource-intensive. Training them requires substantial computational time, careful hyperparameter search, and many design decisions (such as sequence length, network depth, learning rates, and reward shaping) that can significantly influence outcomes. This “design space” creates additional degrees of freedom that are hard to fully document and evaluate. Finally, our models rely primarily on technical indicators derived from price and volume. While this aligns with much of the ML-for-trading literature [10, 8], it omits potentially valuable information from macroeconomic variables, fundamental data, and sentiment indicators, which could change relative performance rankings if incorporated.

4.4 Future Work

The findings suggest several natural extensions. One direction is to build hybrid models that combine the strengths of different families. For example, ARIMA forecasts could be used as additional features for Random Forests, the profit-aligned Neural Network, or the RL agent, allowing these models to condition their decisions on both raw indicators and the output of a strong statistical forecaster. Alternatively, mispricing relative to ARIMA or Random Forest predictions could be used to define mean-reversion or breakout signals that a higher-level policy learns to trade.

Another direction is to enrich the feature space with macroeconomic, fundamental, or sentiment information, following recent work that incorporates earnings, analyst revisions, or news into ML-based stock selection [3, 1]. This could help distinguish between periods when technical signals are likely to be reliable and periods when they are dominated by

news or structural shifts. A third avenue is to refine the reward structure in the RL and profit-aligned neural models to reflect more realistic investor objectives, such as drawdown constraints, risk budgets, or multi-asset portfolio allocation rather than single-stock trading. Finally, a systematic robustness study, testing models under explicit regime splits, stress periods, and alternative transaction-cost assumptions, would provide a clearer view of which methods are truly stable versus those that are primarily backtest artifacts.

5 Conclusions

This project compared classical statistical forecasting, supervised machine learning, and reinforcement learning approaches for modeling and trading U.S. equities under a common data pipeline and evaluation framework. In direct response to our main research question, the results show that reinforcement learning does provide measurable advantages over traditional supervised models on some volatile, trend-driven tickers, but it does not dominate across the board. Linear and Logistic Regression offer interpretability and clarify how technical indicators relate to short-term price continuity, but they provide limited directional or trading edge. Random Forests demonstrate that nonlinear supervised models can achieve extremely high predictive accuracy for next-day prices, yet converting that accuracy into profitable daily trading decisions remains challenging. The profit-aligned Neural Network shows that directly optimizing a PnL-based objective can produce strong and more consistent risk-adjusted performance than our RL agents on average, but at the cost of reduced transparency and potential sensitivity to regime changes and design choices.

Rolling ARIMA confirms that classical time-series models remain highly competitive for one-step-ahead forecasting; in many cases, they perform well enough that complex models must deliver a clear incremental benefit to be worthwhile. Reinforcement learning agents, finally, exhibit the largest upside on certain volatile, trend-driven assets but also the most

variability and downside risk, underscoring the importance of careful reward design, hyperparameter tuning, and external risk controls. Together, these findings support a nuanced view: advanced machine learning and RL techniques can complement, but not fully replace, simpler statistical and supervised methods in practical trading systems.

For practitioners and researchers, the key takeaway is that forecasting accuracy, economic performance, and robustness should be evaluated jointly rather than in isolation. A balanced workflow that combines interpretable baselines, strong statistical benchmarks, and well-designed profit-aligned or reinforcement-based components—embedded within a clear risk-management framework—is more likely to perform reliably than any single model on its own. Future work that explores hybrid architectures, richer information sets, and more rigorous robustness testing can build on this foundation and help clarify when sophisticated models genuinely add value in equity trading and when simpler, well-understood approaches are sufficient.

References

- [1] Doron Avramov, Guy Kaplanski, and Avaniidhar Subrahmanyam. Machine learning vs. economic restrictions: Evidence from stock return predictability. *Management Science*, 2022.
- [2] Fawaz Awad, Fawaz Alqallaf, Mohammed Alsaffar, and Fahad Almutairi. Stock market prediction using deep reinforcement learning. *AI*, 6(6):106, 2023.
- [3] Yi Huang, Luiz Fernando Capretz, and Quang Ho. Machine learning for stock prediction based on fundamental analysis. In *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 01–08, 2021.

- [4] Ahmad Kabbani and Hasan Duman. Deep reinforcement learning approach for trading automation in the stock market. *IEEE Access*, 10:82058–82071, 2022.
- [5] Xiaoyang Li, Jian Ni, and Victor Chang. Application of deep reinforcement learning in stock trading strategies and stock forecasting. *Computing*, 102:1305–1322, 2020.
- [6] John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889, 2001.
- [7] Paul Mooney. Stock market data (nasdaq, nyse, s&p 500). <https://www.kaggle.com/datasets/paultimothymooney/stock-market-data>, 2023.
- [8] Mojtaba Nabipour, Peyman Nayyeri, Hadis Jabani, Saeed Shahab, and Amir Mosavi. Predicting stock market trends using machine learning and deep learning algorithms via continuous and binary data: A comparative analysis. *IEEE Access*, 8:150199–150212, 2020.
- [9] Iqra Mushtaq Parray, Rabia Khaliq, Zahid Ali, Muhammad Waseem Anwar, and Shahrukh Khan. Time series data analysis of stock price movement using machine learning techniques. *Soft Computing*, 24:13409–13421, 2020.
- [10] Jigar Patel, Sahil Shah, Parth Thakkar, and Ketan Kotecha. Predicting stock market index using fusion of machine learning techniques. *Expert Systems with Applications*, 42(4):2162–2172, 2015.
- [11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- [12] Xue Zhong and David Enke. Predicting the daily return direction of the stock market using hybrid machine learning algorithms. *Financial Innovation*, 5(24):1–20, 2019.

Appendix A

Project Code: <https://github.com/jamieth0405/DAT490-Data-Science-Capstone/tree/main>